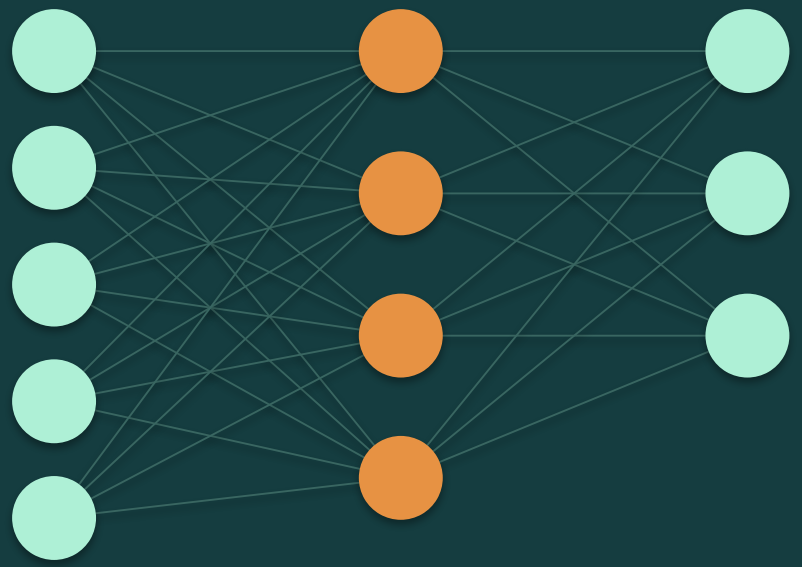
Une IA qui apprend.

Du neurone à l’interface Python.

Comprendre, entraîner et tester un petit réseau de neurones.

DÉBUTANT +

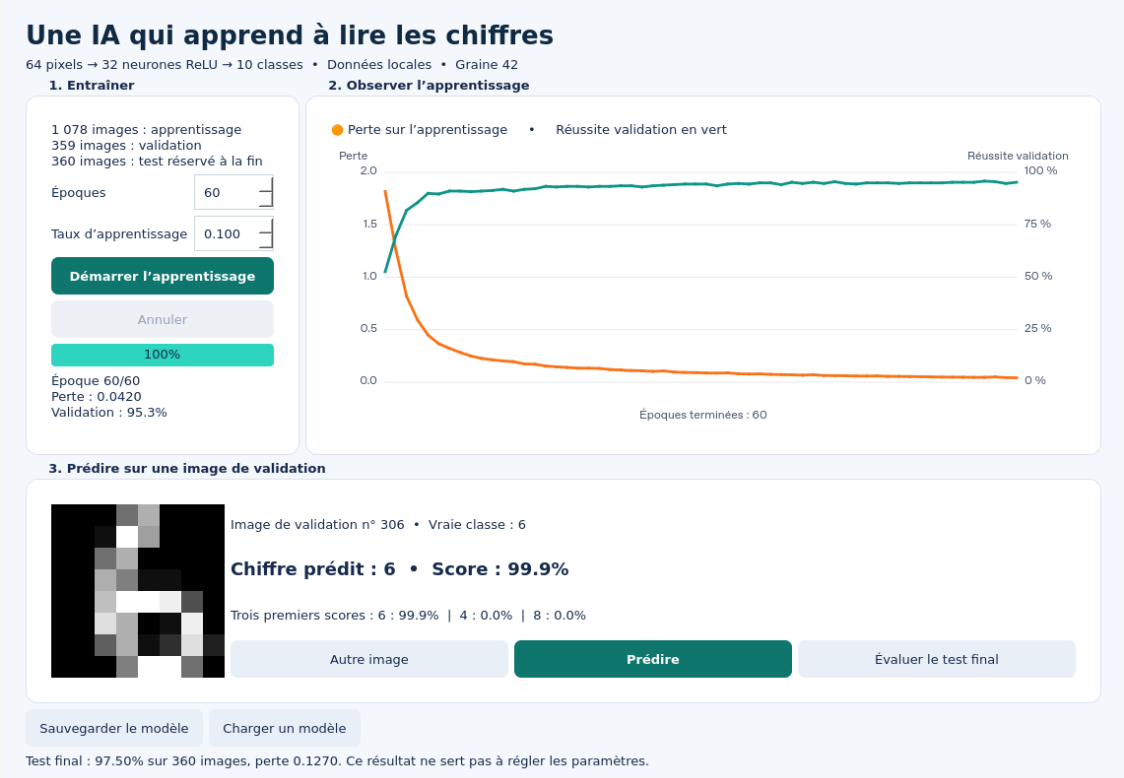
ENVIRON 2 HEURES



64 pixels → 32 neurones → 10 chiffres

Le projet que vous allez construire

Une application qui apprend à classer les chiffres manuscrits de 0 à 9.



01 Apprendre

Ajuster les poids
à partir d'exemples.

02 Prédire

Recevoir une image et
proposer un chiffre.

Votre parcours en quatre étapes

Vous n’avez pas besoin de maîtriser le calcul différentiel pour commencer.

01 Comprendre

Données, neurones et poids.

≈ 25 min

02 Programmer

Propagation, erreur et mise à jour.

≈ 35 min

03 Observer

Courbes, validation et petit écran Qt.

≈ 25 min

04 Expérimenter

Exercices, erreurs et bilan final.

≈ 35 min

Objectif : savoir expliquer ce qui change quand le modèle « apprend ».

Apprendre = ajuster des nombres

Le programme connaît la méthode. Les exemples déterminent les poids.

Entrée x

Une image de chiffre.

64 valeurs numériques.

→

Modèle fθ

Des poids θ.

Une transformation calculée.

→

Sortie ŷ

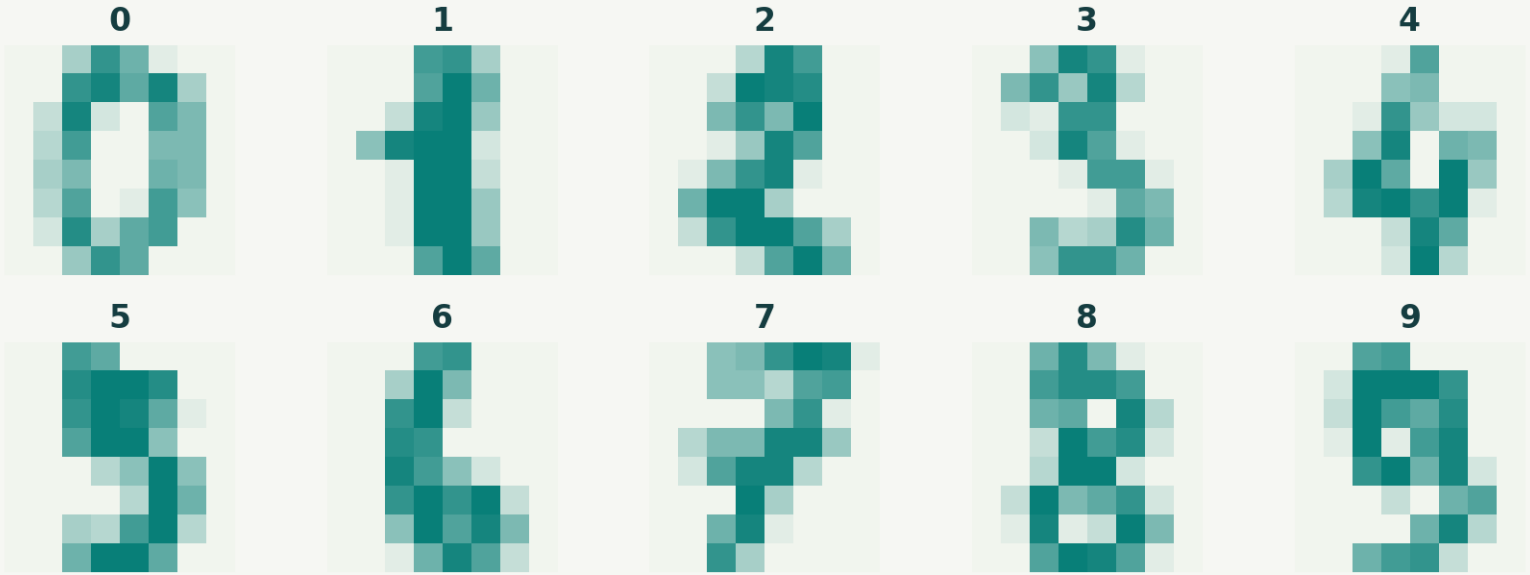
10 scores normalisés.

Le chiffre le plus probable.

L’étiquette y permet de mesurer l’erreur entre la réponse attendue et la prédiction.

Notre matière première : 1 797 images

load_digits() fournit un petit jeu de données directement avec scikit-learn.



À retenir

8 × 8 pixels

10 classes : 0 à 9

Valeurs : 0 à 16

Source : scikit-learn, load_digits — jeu de chiffres manuscrits 8×8.

Trois ensembles, trois rôles distincts

On sépare les données avant de commencer à entraîner le réseau.



1078 images

Apprendre

Mettre les poids à jour.

359 images

Choisir

Comparer les réglages.

360 images

Évaluer

Mesurer une fois à la fin.

Même graine, mêmes ensembles. Aucun exemple ne doit apparaître dans deux ensembles.

De l’image au vecteur de nombres

Une transformation simple, identique pendant l’entraînement et la prédiction.

Un chiffre = une grille de 8 × 8 intensités

0	0	7	8	13	16	15	1
0	0	7	7	4	11	12	0
0	0	0	0	8	13	1	0
0	4	8	8	15	15	6	0
0	2	11	15	15	4	0	0
0	0	0	16	5	0	0	0
0	0	9	15	1	0	0	0
0	0	13	5	0	0	0	0

PRÉPARATION DES ENTRÉES

```
x = digits.data.astype(np.float64) / 16.0
y = digits.target.astype(np.int64)
```

Une image : (64,)

Un lot de 32 images : (32, 64)

Un neurone : pondérer, additionner, activer

Chaque poids règle l'influence d'une entrée ; le biais décale le résultat.

Somme pondérée

$z = w_1x_1 + w_2x_2 + b$

Exemple :

$2 \times 0{,}3 - 0{,}4 + 0{,}1 = 0{,}3$

→

Activation ReLU

$a = \max(0, z)$

$z = 0{,}3 \rightarrow a = 0{,}3$

$z = -0{,}8 \rightarrow a = 0$

ReLU ajoute une non-linéarité : le réseau peut représenter des relations plus complexes.

Notre réseau : 64 → 32 → 10

Une couche cachée suffit pour ce premier projet pédagogique.

2 410

paramètres

à apprendre

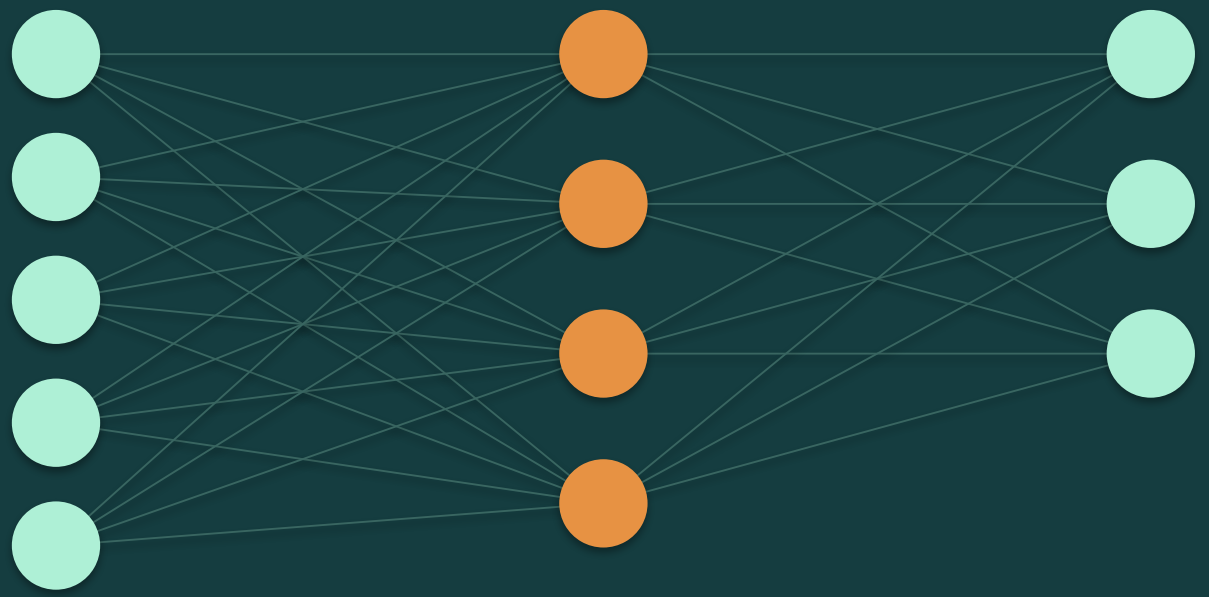


Schéma simplifié : tous les neurones ne sont pas représentés.

Calculer une prédiction, par lots

NumPy fait les multiplications de matrices avec l'opérateur @.

Entrées

X

B × 64

Couche cachée

ReLU(X @ W1 + b1)

B × 32

Sorties

softmax(H @ W2 + b2)

B × 10

```
W1.shape == (64, 32)     W2.shape == (32, 10)
```

Softmax et erreur : deux rôles différents

Une distribution pour prédire ; une fonction de coût pour apprendre.

Softmax

$p_k = \exp(z_k) / \Sigma \exp(z_j)$

Les 10 probabilités totalisent 1.

La prédiction choisit le maximum.

Entropie croisée

Pour le vrai chiffre y : $-\ln(p_y)$

$p_y = 0{,}8 \rightarrow$ erreur ≈ 0,223

$p_y = 0{,}1 \rightarrow$ erreur ≈ 2,303

Le but est de réduire l'erreur ; le pourcentage de bonnes réponses mesure autre chose.

Apprendre : faire un petit pas dans le bon sens

Le gradient indique comment une variation des poids change l’erreur.

$$\text{poids} \leftarrow \text{poids} - \text{taux} \times \text{gradient}$$

Le poids

Un nombre appris.

Le taux η

La taille de chaque pas.

Le gradient

La direction locale.

Exemple : 0,5 − 0,1 × 0,2 = 0,48. Le même principe s’applique à tous les paramètres.

La rétropropagation, vue de près

Option mathématique : propager la sensibilité de l'erreur vers l'entrée.

NETWORK.LOSS_AND_GRADIENTS — EXTRAIT

```
dz2 = np.exp(log_probs)
dz2[np.arange(len(y)), y] -= 1
dz2 /= len(y)
dw2 = hidden.T @ dz2
dz1 = (dz2 @ self.w2.T) * (z1 > 0)
dw1 = x.T @ dz1
```

1. Erreur en sortie
2. Gradient de W2
3. Traversée de ReLU
4. Gradient de W1

Les biais ont aussi un gradient : somme des gradients du lot, colonne par colonne.

La boucle d’entraînement

Un lot = quelques exemples. Une époque = un passage sur tout le train.

1. Mélanger

Changer l’ordre des exemples.

2. Prédire

Calculer la sortie pour un lot.

3. Corriger

Calculer les gradients et ajuster les poids.

4. Mesurer

Suivre train + validation après chaque époque.

Réglages du projet : 60 époques · lots de 32 · taux 0,1 · graine 42.

Installer et lancer le projet

Exemple Windows / PowerShell, depuis le dossier code du projet décompressé.

TROIS COMMANDES

```
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe app.py
```

NumPy : calculs • scikit-learn : données et séparation • PySide6 : interface

Versions du projet : NumPy 2.2.6 · scikit-learn 1.7.2 · PySide6 6.8.3. Installation desktop requise pour afficher la fenêtre.

Séparer le modèle de son interface

On peut entraîner le réseau sans ouvrir de fenêtre.

model.py

Les données

Le réseau NumPy

L’apprentissage

La sauvegarde

app.py

La fenêtre Qt

Les contrôles

Le travail en arrière-plan

L’affichage des résultats

test_model.py

Vérifier les gradients

Mesurer les progrès

Tester la sauvegarde

Contrôler les ensembles

Cette architecture facilite la compréhension, les tests et les futures améliorations.

Préparer les ensembles dans Python

stratify conserve les proportions des chiffres ; random_state fixe le tirage.

LOAD_DATA — EXTRAIT AVEC GRAINE EXPLICITE

```
train, rest = train_test_split(
    np.arange(len(y)), test_size=0.4,
    stratify=y, random_state=42
)
val, test = train_test_split(
    rest, test_size=0.5,
    stratify=y[rest], random_state=43
)
```

Résultat mesuré : train 1078 · validation 359 · test 360.

La propagation avant tient en sept lignes

Voici le cœur de Network.forward(), utilisé pour apprendre et pour prédire.

EXTRAIT EXACT DE MODEL.PY

```
z1 = x @ self.w1 + self.b1
hidden = np.maximum(0, z1)
logits = hidden @ self.w2 + self.b2
shifted = logits - logits.max(axis=1, keepdims=True)
log_probs = shifted - np.log(
    np.exp(shifted).sum(axis=1, keepdims=True))
return z1, hidden, log_probs
```

La stabilité numérique fait partie de l’algorithme : elle évite de calculer exp(1000).

Entraîner, puis regarder la validation

Les réglages sont explicites pour pouvoir refaire la même expérience.

BOUCLE MINIMALE COMPATIBLE AVEC MODEL.PY

```
data = load_data(seed=42)
model = Network(seed=42)
rng = np.random.default_rng(42)
for epoch in range(60):
    model.train_epoch(
        data["x_train"], data["y_train"],
        lr=0.1, batch_size=32, rng=rng)
    val = model.evaluate(
        data["x_val"], data["y_val"])
    print(epoch + 1, val)
```

Train :

modifie les poids.

Validation :

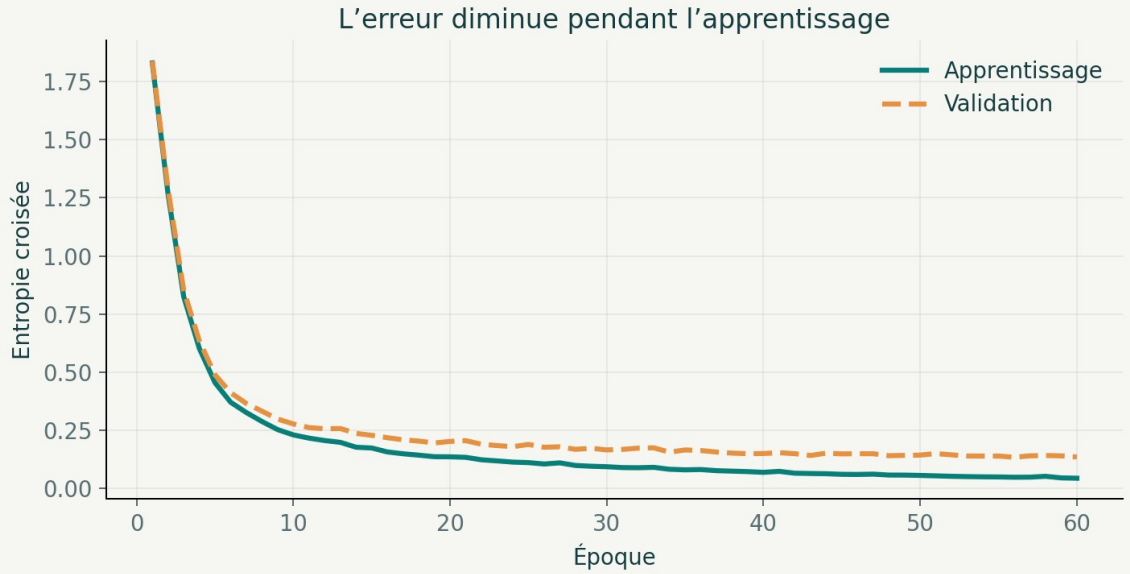
mesure seulement.

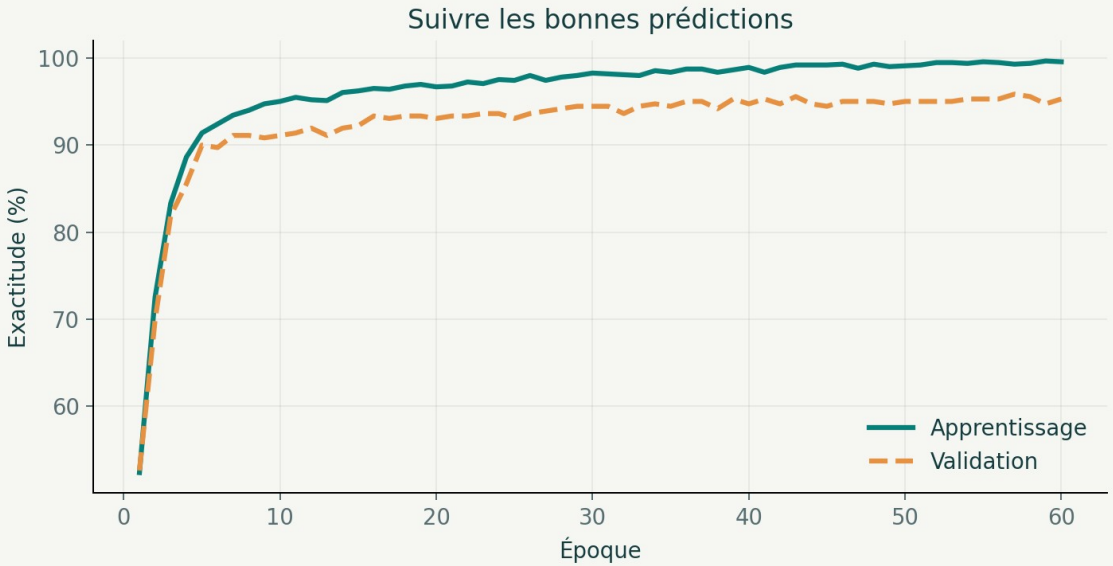
Test :

attend la fin.

Lire les courbes d’apprentissage

Résultats réellement obtenus avec le code fourni, sur une exécution reproductible.





À 60 époques : train 99,5 % / validation 95,3 %.

Source : expérience locale mesurée ; paramètres complets et 60 points par courbe dans support/metrics.json.

Une bonne courbe peut cacher un problème

Le score d’apprentissage seul ne dit pas si le réseau généralise.

Pas assez appris

Train faible
Validation faible

Vérifier les données,
le taux et les époques.

Trop mémorisé

Train élevé
Validation dégradée

Arrêter plus tôt ou
régulariser le modèle.

Évaluation faussée

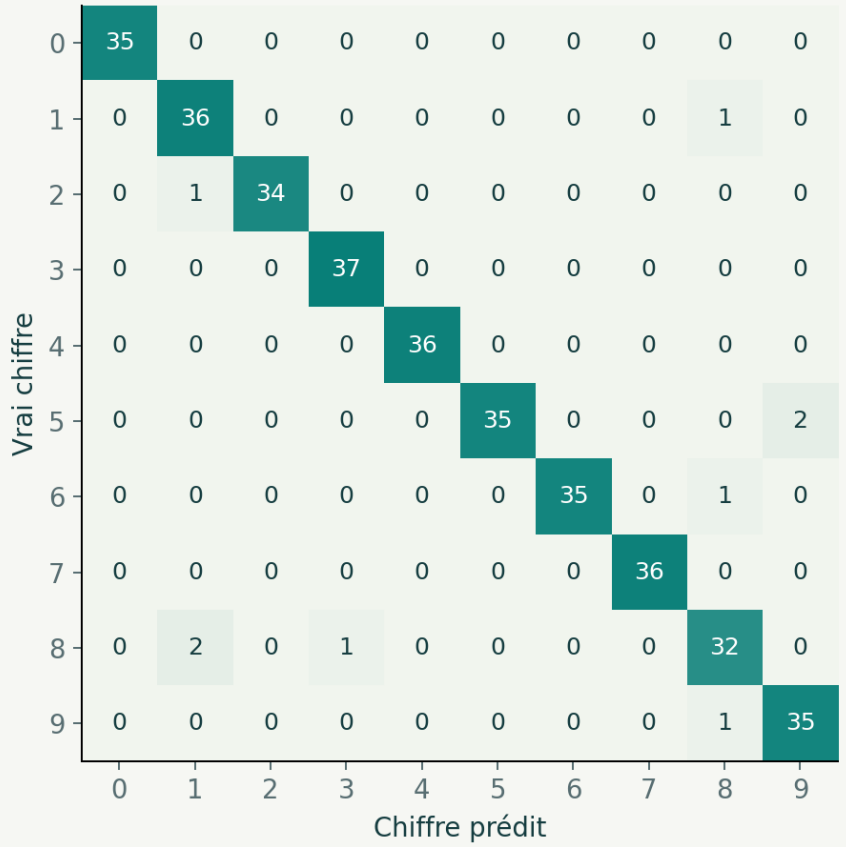
Des données se croisent
ou le test guide les choix.

Repartir d’une séparation
correcte des rôles.

Le bon réflexe : conserver une validation indépendante et garder le test pour le bilan final.

Le bilan final sur des images réservées

La matrice de confusion permet de voir quels chiffres sont confondus.



97,5 %

de bonnes réponses

sur 360 images de test.

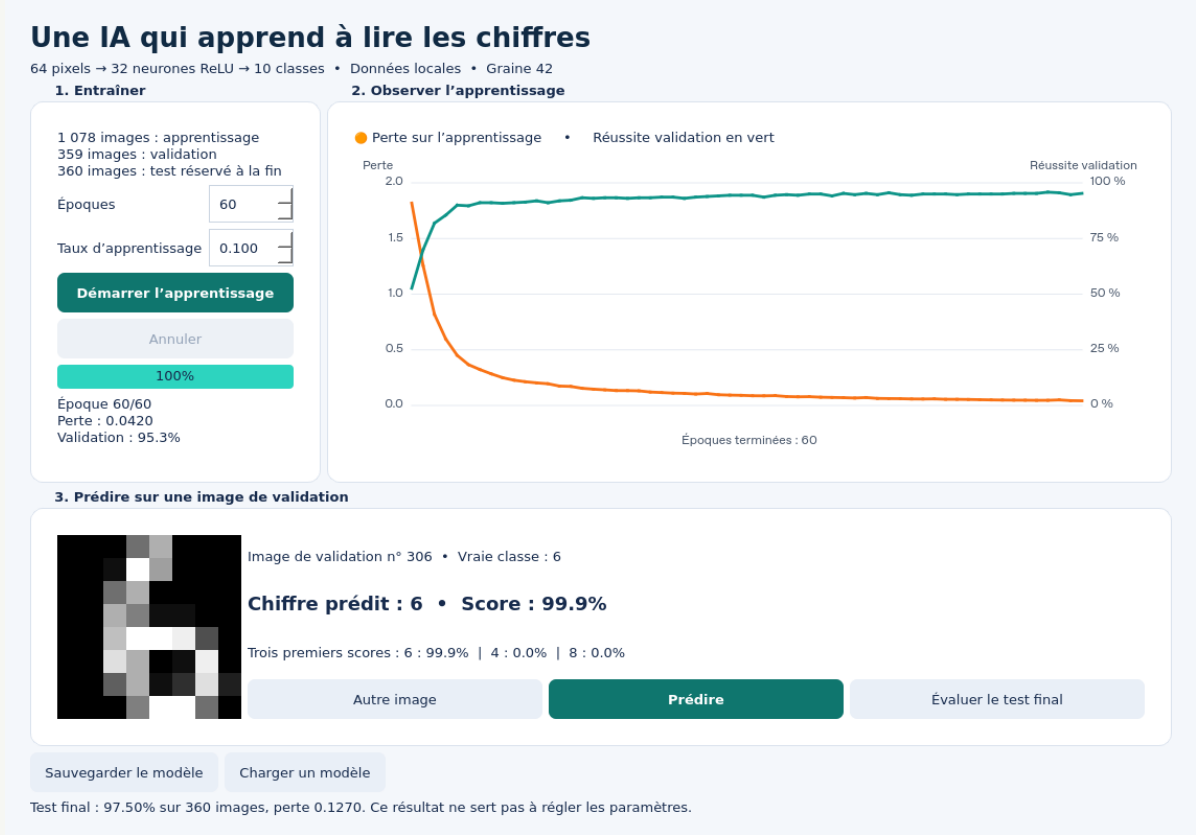
Diagonale = correct

Hors diagonale = confusion

Mesure locale : 64→32→10 · SGD · 60 époques · taux 0,1 · lots 32 · graine 42. Score non garanti sur d'autres données.

Une petite interface, quatre actions

L'écran rend l'apprentissage observable sans masquer le fonctionnement du modèle.



1 Choisir les réglages

2 Lancer l'entraînement

3 Prédire un exemple

4 Sauvegarder le modèle

Garder la fenêtre réactive

L’entraînement tourne dans un worker ; l’interface reçoit des signaux Qt.

Thread de l’interface

Boutons et contrôles

Courbes et résultats

Réception des signaux

→

←

Worker d’entraînement

Boucle sur les époques

Gradients et mises à jour

Émission des métriques

Le worker calcule ; les slots de la fenêtre mettent les widgets à jour.

Source : Qt for Python, QThread et mécanisme de signaux/slots.

Relier les calculs aux actions utilisateur

Une action a une responsabilité précise : entraîner, annuler ou prédire.

Entraîner	Créer un modèle et démarrer le worker.
Progression	Afficher la perte et l'exactitude de validation.
Annuler	Demander l'arrêt, puis attendre la fin du worker.
Prédire	Calculer predict_proba(image) puis argmax.
Évaluer le test	Calculer une mesure finale, sans gradient.

Sauvegarder ce que le réseau a appris

Un modèle entraîné est principalement un ensemble de poids et de biais.

API DU MODÈLE FOURNI

```
model.save("mon_modele.npz")
model = Network.load("mon_modele.npz")
probas = model.predict_proba(image)
```

À conserver ensemble

Les poids

L'architecture

La normalisation

L'ordre des classes

Prédire ne modifie pas les poids. Le modèle ne continue pas à apprendre tout seul.

À vous : trois expériences courtes

Notez votre hypothèse avant de lancer le programme. Gardez le test fermé.

A Compter

Imaginez 16 neurones cachés.

Combien de poids
et de biais ?

B Comparer

20 époques, seed 42.
Taux : 0,01 puis 0,1.

Quel apprentissage
semble le plus efficace ?

C Diagnostiquer

Scénario fictif :
train = 99,8 %
validation = 86 % et baisse.

Que changer en premier ?

Exercice code : retrouver les trois choix

Recréez le classement affiché par l’interface, puis comparez avec predict_sample().

POINT DE DÉPART

```
probas = model.predict_proba(image)[0]
# 1. Trouver les 3 plus grands scores.
# 2. Retrouver leurs chiffres.
# 3. Formater les résultats en pourcentages.
```

Critère de réussite

Trois chiffres distincts,
triés par score décroissant.

Les poids ne changent pas.

Indice : np.argsort renvoie les indices du plus petit score au plus grand.

Corrections : expliquer le résultat

Le raisonnement compte autant que le score obtenu.

A 64 × 16 + 16 + 16 × 10 + 10
= 1 210 paramètres.

B Comparer les pertes de validation.

C Surapprentissage probable :
surveiller la validation et arrêter plus tôt.

CORRECTION DE L'EXERCICE D

```
top3 = np.argsort(probas)[-3:][::-1]
for chiffre in top3:
    score = 100 * probas[chiffre]
    print(f"{chiffre} : {score:.1f} %")
```

Une explication honnête distingue le score mesuré, une hypothèse et une garantie.

Quand cela ne fonctionne pas

Commencez par le symptôme, puis vérifiez une seule hypothèse à la fois.

Module introuvable	Installer requirements.txt avec le Python de .venv.
Erreur ou valeurs NaN	Baisser le taux ; vérifier les nombres et les dimensions.
L'interface se bloque	Déplacer la boucle d'entraînement dans le worker.
Les réponses sont mauvaises	Vérifier /16, forme (1,64), ordre des pixels et des classes.
Aucune fenêtre dans le cloud	Lancer sur un bureau ; offscreen sert uniquement à tester.

Ce que vous avez réellement construit

Un classifieur de chiffres, avec une vraie boucle d’apprentissage.

Vous savez expliquer

Les poids, l’erreur et la mise à jour.

Vous savez vérifier

Un gradient, des courbes et un score indépendant.

Vous savez relier

Un modèle numérique à une interface réactive.

Prochaine étape : ajouter l’arrêt anticipé en gardant les meilleurs poids de validation.

Ressources et fichiers du cours

Le PowerPoint est éditable ; les notes de chaque diapositive détaillent les explications.

Dans le dossier du cours

code/ : application et tests

support/ : figures et mesures

livrables/ : PowerPoint et PDF

LIRE_MOI.md : démarrage rapide

NumPy — prise en main ↗️

scikit-learn — load_digits ↗️

scikit-learn — train_test_split ↗️

Qt for Python — QThread ↗️

scikit-learn — réseaux de neurones ↗️